The Offline SQL Mastery Bank: 150 Practice Scenarios

150 distinct, hand-crafted business requirements across 15 core analytical categories.

Core Query Structure

Q1: Retrieve all columns from the `employees` table for employees hired after 2020.

Q2: Select only the `first_name`, `last_name`, and `email` from the `customers` table.

Q3: List all unique `job_titles` currently present in the `employees` table using DISTINCT.

Q4: Retrieve the top 5 highest-paid employees by ordering the `salary` column in descending order.

Q5: Select all rows from `products`, but alias the `retail_price` column as `price_usd`.

Q6: Retrieve the first 10 rows from the `transactions` table ordered by `transaction_date`.

Q7: Select `employee_id` and calculate a new column `annual_bonus` as 10% of `base_salary`.

Q8: List all `orders` sorted by `store_id` ascending, and then by `order_date` descending.

Q9: Retrieve `product_name` and a hardcoded string 'Active' as a new column named `status`.

Q10: Select the `order_id` and the total price (quantity * unit_price) aliased as `total_value`.

Filtering Operators

Q1: Find all transactions in the `sales` table where the `amount` is strictly greater than 500.

Q2: Retrieve all customers whose `last_name` starts with the letter 'S' using the LIKE operator.

Q3: Filter the `orders` table for purchases made between '2023-01-01' and '2023-12-31'.

Q4: Find all employees who work in `department_id` 1, 3, or 5 using the IN operator.

Q5: Retrieve products where the `category` is 'Electronics' AND the `price` is less than 50.

Q6: List all active users (status = 'Active') OR users who logged in within the last 7 days.

Q7: Find all records in the `inventory` table where `restock_date` IS NULL.

Q8: Retrieve all vendors whose `company_name` contains the word 'Tech' anywhere in the string.

Q9: Filter out all canceled orders using the != or <> operator on the `status` column.

Q10: Find customers located in 'New York' whose `account_balance` is NOT BETWEEN 1000 and 5000.

Aggregation and Grouping

Q1: Calculate the total (SUM) revenue generated across all completed orders.

Q2: Count the total number of unique customers who made a purchase this month.

Q3: Find the minimum, maximum, and average salary for all employees in the database.

Q4: Group by `department_id` and calculate the total headcount (COUNT) for each department.

Q5: Calculate the average order value (AVG) grouped by `product_category`.

Q6: Group sales by `store_location` and only show locations with total revenue exceeding $10,000 (HAVING).

Q7: Find the total number of items sold per `vendor_id` and order the results from highest to lowest.

Q8: Count how many employees share the same `job_title`, excluding titles with fewer than 3 employees.

Q9: Group by the extract of the YEAR from `order_date` to find yearly total revenue.

Q10: Calculate the average rating per `product_id`, rounding the result to two decimal places.

Joins

Q1: Write an INNER JOIN to combine `users` and `orders` on `user_id` to show order history.

Q2: Use a LEFT JOIN to list all `products` and any associated `reviews` (including products with no reviews).

Q3: Perform a RIGHT JOIN between `departments` and `employees` to find departments with no staff.

Q4: Write a FULL OUTER JOIN to find mismatched records between two legacy `customer` tables.

Q5: Join `order_items`, `orders`, and `products` to display order ID, product name, and quantity.

Q6: Use a SELF JOIN on the `employees` table to list employees alongside their manager's name.

Q7: Join `sales` and `date_dim` tables, filtering the result for weekend sales only.

Q8: Write a CROSS JOIN between `colors` and `sizes` tables to generate all possible product variations.

Q9: Use an anti-join (LEFT JOIN + IS NULL) to find users who registered but never placed an order.

Q10: Join two tables using multiple conditions (e.g., matching both `store_id` and `region_id`).

Conditional Logic

Q1: Use a CASE statement to label orders > $1000 as 'High Value' and others as 'Standard'.

Q2: Write a query grouping `age` into buckets: '<18', '18-35', '36-50', and '50+' using CASE.

Q3: Use CASE inside a SUM function to count how many orders have a 'Shipped' status.

Q4: Create a priority flag using CASE: If `status`='Urgent' THEN 1, IF 'Normal' THEN 2, ELSE 3.

Q5: Use a CASE statement in the ORDER BY clause to sort 'Premium' members to the top of the list.

Q6: Write a conditional check to flag an employee as 'Eligible for Bonus' if their tenure > 3 years.

Q7: Use IIF or a simple CASE to return 'Yes' if a `discount_code` was applied, otherwise 'No'.

Q8: Categorize products based on stock: 'Out of Stock' (0), 'Low Stock' (<10), and 'In Stock' (>=10).

Q9: Use a CASE statement to apply different tax rates based on the `state_code` of the transaction.

Q10: Update a boolean integer column: return 1 if `is_active` is true and `last_login` < 30 days, else 0.

Basic Data Type Conversion

Q1: CAST a string `order_date` ('2023-01-15') into a standard SQL DATE data type.

Q2: CONVERT a floating-point `total_weight` into an INTEGER to remove decimal precision.

Q3: Cast the `employee_id` (integer) to a VARCHAR so it can be concatenated with a prefix.

Q4: Convert a Unix timestamp integer into a readable DATETIME format.

Q5: Use safe casting (e.g., TRY_CAST) to handle alpha-numeric characters in a supposed numeric column.

Q6: CAST a numeric percentage (0.15) to a string and append a '%' symbol to it.

Q7: Convert a DATETIME column to purely a DATE column to strip away the time metadata.

Q8: Change a string 'true'/'false' column into a standard boolean/BIT data type.

Q9: Format a standard DATE column into a specific string format (e.g., 'MM-DD-YYYY').

Q10: Cast two integers as floats before dividing them to prevent integer division (truncation).

Window Functions

Q1: Use ROW_NUMBER() to rank employees by salary within each specific `department_id`.

Q2: Calculate the running total of sales for the year using SUM() OVER(ORDER BY order_date).

Q3: Find the day-over-day revenue difference using the LAG() function.

Q4: Use LEAD() to find the date of the *next* purchase made by a specific customer.

Q5: Rank products by total sales using RANK() and DENSE_RANK() to see how ties are handled.

Q6: Use NTILE(4) to divide customers into quartiles based on their lifetime spend.

Q7: Calculate a 7-day moving average of sales using AVG() OVER with a ROWS BETWEEN clause.

Q8: Find the highest salary in the company and display it next to every row using MAX() OVER().

Q9: Use FIRST_VALUE() to find the name of the very first product ever purchased by each user.

Q10: Calculate the percentage contribution of each region to total sales using Window Functions.

Common Table Expressions (CTEs)

Q1: Write a basic CTE to filter active employees, then select their names from the CTE.

Q2: Create two CTEs (one for sales, one for returns) and join them together in the final query.

Q3: Use a CTE to calculate average salary per department, then find departments above the company average.

Q4: Write a CTE to deduplicate rows by assigning a ROW_NUMBER, then delete rows where rn > 1.

Q5: Use a recursive CTE to build a hierarchical organizational chart (Manager -> Employee).

Q6: Create a CTE that defines a static list of dates, and join it to sales to find days with zero revenue.

Q7: Use a CTE to find the top 3 customers, then query their full demographic details in the main query.

Q8: Write a CTE to extract complex JSON/XML data, then apply basic filtering in the main SELECT.

Q9: Calculate customer lifetime value in a CTE and use it to group users into VIP tiers.

Q10: Refactor a complex nested subquery into a readable CTE format.

Subqueries

Q1: Use a subquery in the WHERE clause to find employees earning more than the overall company average.

Q2: Write an IN subquery to find all `orders` placed by customers located in 'California'.

Q3: Use an EXISTS clause with a correlated subquery to find users who have at least one active subscription.

Q4: Write a subquery in the SELECT clause to display the total company revenue next to individual order rows.

Q5: Use a subquery in the FROM clause (derived table) to pre-aggregate daily sales before joining.

Q6: Find the product with the maximum price using a WHERE `price` = (SELECT MAX(price)...) subquery.

Q7: Write a NOT EXISTS query to find all active promotional codes that have never been used.

Q8: Use a correlated subquery in the HAVING clause to filter groups based on a dynamic threshold.

Q9: Find the second highest salary in the `employees` table using a subquery and MAX().

Q10: Update a table based on aggregated values calculated in a subquery.

Set Operations

Q1: Use UNION to combine lists of `emails` from a `b2b_customers` table and a `b2c_customers` table.

Q2: Use UNION ALL to merge Q1 and Q2 sales data tables without removing any duplicate rows.

Q3: Find the common `product_id`s present in both 'Cart' and 'Wishlist' tables using INTERSECT.

Q4: Use EXCEPT (or MINUS) to find employees in the `hr_roster` who are missing from the `payroll` table.

Q5: Combine the names of `vendors` and `customers` into a single column using UNION, padding with a 'Type' column.

Q6: Use UNION ALL to unpivot three columns (Jan_Rev, Feb_Rev, Mar_Rev) into a single 'Month' column.

Q7: Find elements unique to the first table by using EXCEPT, then elements unique to the second, and UNION them.

Q8: Merge historical archived data with live data using UNION ALL for year-over-year reporting.

Q9: Use INTERSECT on a composite key (user_id, product_id) between 2022 and 2023 sales tables.

Q10: Write a complex query using UNION ALL wrapped in a CTE, followed by an aggregation on the result.

Data Manipulation and Definition (DML/DDL)

Q1: Write a CREATE TABLE statement for `inventory` with columns for ID, name, quantity, and a primary key.

Q2: Add a new column named `last_login_date` of type DATETIME to the existing `users` table.

Q3: Write an INSERT statement to add three new specific rows into the `departments` table.

Q4: Use INSERT INTO ... SELECT to copy all inactive accounts into an `archived_accounts` table.

Q5: Write an UPDATE statement to increase the `base_salary` of all 'Engineers' by 5%.

Q6: Update the `status` of orders to 'Overdue' if the `delivery_date` has passed and `status` is 'Pending'.

Q7: Write a DELETE statement to remove all rows from `web_logs` that are older than 90 days.

Q8: Use TRUNCATE to quickly wipe all data from a temporary staging table.

Q9: Create a Non-Clustered Index on the `email` column of the `users` table to speed up lookups.

Q10: Drop a column `temporary_notes` from the `customer_feedback` table safely.

Date and Time Manipulation

Q1: Use CURRENT_DATE or GETDATE() to find all orders placed exactly today.

Q2: Use a DATEDIFF function to calculate the number of days a ticket was open (`resolved_date` - `created_date`).

Q3: Add exactly 30 days to the `invoice_date` using DATEADD to calculate the `due_date`.

Q4: Extract only the MONTH and YEAR from a timestamp column to aggregate monthly active users.

Q5: Use DATE_TRUNC to round a precise timestamp down to the nearest hour.

Q6: Find the last day of the current month using LAST_DAY() or equivalent date math.

Q7: Filter for records where the `birth_date` occurred on a weekend (Saturday or Sunday).

Q8: Calculate a user's exact age in years based on their `dob` and the current date.

Q9: Find the overlapping days between two different date ranges (start1, end1 and start2, end2).

Q10: Format a raw datetime column into 'Day-MonthName-Year' (e.g., '15-August-2023') for a report.

String Manipulation

Q1: Concatenate `first_name` and `last_name` with a space in between to create a `full_name` column.

Q2: Use SUBSTRING to extract the first 3 characters of a `phone_number` to find the area code.

Q3: Use UPPER and LOWER to standardize all entered email addresses to lowercase.

Q4: Find and REPLACE all instances of 'http://' with 'https://' in the `website_url` column.

Q5: Use TRIM to remove accidental leading and trailing spaces from user-entered `usernames`.

Q6: Find the position/index of the '@' symbol in an email address using CHARINDEX or POSITION.

Q7: Extract everything after the '@' symbol to group users by their email domain.

Q8: Pad a numeric `employee_id` with leading zeros (e.g., '00045') using LPAD or string math.

Q9: Split a comma-separated string 'Red,Green,Blue' and extract the second element.

Q10: Calculate the LENGTH of passwords and flag any that are shorter than 8 characters.

Handling Nulls

Q1: Use COALESCE to return 'N/A' whenever the `phone_number` column is NULL.

Q2: Replace NULL `bonus_amount` values with 0 using IFNULL before adding them to the salary.

Q3: Write a WHERE clause to exclusively select rows where the `assigned_manager` IS NOT NULL.

Q4: Understand aggregate behavior: Write a query showing the difference between COUNT(*) and COUNT(column_name).

Q5: Use COALESCE inside a JOIN condition to match records where keys might be missing.

Q6: Use NULLIF to divide total sales by the number of complaints, preventing a divide-by-zero error.

Q7: Sort a list of tasks by `due_date`, ensuring all tasks with NULL due dates appear at the very end.

Q8: Filter a dataset to find rows where all three specific columns (`email`, `phone`, `address`) are NULL.

Q9: Use CASE to create a boolean flag: 1 if `discount` IS NULL, else 0.

Q10: Perform string concatenation safely with NULLs (ensuring the whole string doesn't become NULL).

Views and Materialized Views

Q1: Create a standard VIEW called `vw_active_staff` that hides terminated employees from basic users.

Q2: Write a CREATE OR REPLACE VIEW statement to add a new calculated column to an existing view.

Q3: Create a view that joins `orders`, `customers`, and `products` to serve as a BI reporting layer.

Q4: Query a system dictionary/catalog to find all views currently existing in the database.

Q5: Write a VIEW that restricts row access (e.g., only showing data for `region = 'North'`).

Q6: Create a Materialized View for a highly complex daily sales aggregation to improve dashboard speed.

Q7: Write the specific command required to REFRESH a Materialized View after a data load.

Q8: Drop an obsolete view named `vw_2020_archived_metrics`.

Q9: Create a view that utilizes a CTE and Window Functions to expose a pre-calculated 'Ranking' metric.

Q10: Update a base table and write a query against a view to observe how standard views reflect real-time changes.